

Integrating 4D FMCW Radar With Multi-Sensor System For Railway Ego-Velocity Estimation

Author:

Martyn Cox

Student Number: c3374433

Supervisors:

Alex Biddulph, Aaron Wong - 4AI Systems

Thesis

submitted in partial fulfilment of the requirements for the Degree of
Bachelor of Software Engineering (Honours)



School of Information and Physical Sciences
The University of Newcastle
Callaghan, New South Wales 2308, Australia
November 2024

Acknowledgments

I would like to express my gratitude to those who have supported and guided me throughout the course of this research project. I especially extend my appreciation to 4AISystems for partnering with me on this project. I am grateful to my supervisors Aaron and Alex for your continuous support and encouragement. Thanks also to the the entire 4AI team for their guidance and knowledge shared.

Contents

List of Figures	vi
List of Tables	vii
1 Introduction	1
1.1 Background	1
1.2 Problem statement	3
1.3 Objectives	4
1.4 Scope	4
1.5 Significance	5
1.6 Outline	5
2 Literature Review	6
2.1 Methodology	6
2.2 Research Findings	7
3 Methodology and Design	13
3.1 Research Design	13
3.2 Data Collection	14
3.2.1 Low-Velocity Prototype	14
3.2.2 High-Velocity Prototype	16
3.2.3 RadarScenes Dataset	17
3.3 Data Analysis	17
3.4 Validation and Verification	18
3.4.1 Uncertainty Quantification	19
3.5 Design Specifications	19
3.6 Software Development Methodology	20
3.7 Ethical Considerations	21

4	Implementation and Evaluation	23
4.1	Development Process	23
4.1.1	Languages	23
4.1.2	Frameworks and Libraries	24
4.1.3	Hardware	24
4.1.4	Development Environment	25
4.1.5	Coding Practices	25
4.2	Implementation Details	25
4.2.1	Radar Configuration	25
4.2.2	SMSTP Packet Parser	26
4.2.3	Point Cloud Visualisation	27
4.2.4	Least-Squares Radar Velocity Estimation	28
4.2.5	RANSAC Outlier Rejection	29
4.2.6	Cauchy Robust Loss	30
4.3	Challenges and Solutions	32
4.4	Evaluation Methods	33
5	Results	35
5.1	Low-Velocity Testing	35
5.2	RadarScenes Simulation	37
5.3	Contributions and Literature Comparison	39
5.4	Limitations	39
6	Conclusions	41
	References	43
	Appendices	i
A	Performance Data	ii
B	Protocol Buffer Messages	iii
B.1	Point Cloud	iii
B.2	External Body Velocity	iv

List of Figures

2.1	Range and Velocity measurement with FMCW radar	8
3.1	Low-velocity prototype architecture	15
3.2	Low-velocity prototype implementation	16
3.3	System architecture diagram	20
4.1	SMSTP state diagram	26
4.2	Point cloud visualisation	28
4.3	Cauchy loss function for different scaling factors	31
5.1	Low-velocity prototype results	36
5.2	RadarScenes sequence 10 chart with RANSAC with Cauchy velocity estimates	38

List of Tables

3.1	Selected RadarScenes environmental characteristics	17
3.2	Monte Carlo simulation results for ego-velocity estimation algorithms	19
5.1	RadarScenes performance metrics across estimation algorithms	37
A.1	 radar performance data [1]	ii

Abstract

Accurate ego-velocity estimation is crucial for reliable driver advisory systems, particularly in environments where traditional odometry methods perform poorly. Frequency-modulated continuous-wave (FMCW) radar offers an advantage over conventional sensors in its robustness against adverse environmental conditions, such as fog, rain, darkness, and poor satellite signal. This study explores the integration of 4D FMCW radar with an existing multi-sensor system to improve odometry accuracy in rail vehicles. A systematic evaluation of radar-based ego-velocity algorithms is conducted, and a novel approach is proposed, combining random sample consensus (RANSAC) and robust loss techniques. A low-velocity experiment and a simulation based on the RadarScenes dataset were performed to evaluate the implemented velocity estimation approaches. The novel approach was found to provide no significant improvement over conventional radar-based methods. However, the radar ego-velocity estimation was found to be substantially superior to odometry methods of the existing system, particularly at low velocities. These findings underscore the potential of FMCW radar to improve railway driver advisory systems, providing high accuracy across diverse operating conditions.

1

Introduction

Accurate and reliable odometry is essential for railway vehicle safety systems. Radar technology offers the advantage of remaining functional in environments where traditional sensors may struggle. This project explores the potential of integrating frequency-modulated continuous-wave (FMCW) radar with an existing sensor system to improve ego-velocity estimation in a rail environment.

1.1 Background

Recent improvements in Doppler radar technology have led to the growing use of radar sensors in the automotive industry. Doppler radars have become ubiquitous in advanced driver-assistance systems, and have found applications in rail, naval, and aerial vehicles. In particular, FMCW radar systems have garnered much interest due to their low power consumption, cost-effectiveness, and small size [2]. In contrast to traditional pulsed radar, which alternates between transmitting and receiving, FMCW radar continuously emits a waveform and listens for return signals simultaneously.

Radar sensors are able to provide rich information which typically cannot be obtained from other sensors. FMCW radar is able to measure the distance, direction, radial velocity, and radar cross-section (RCS) of objects. These characteristics lead to many applications

for FMCW radar in localisation, object detection, and odometry [3]. In these areas, FMCW radar has provided some benefits over other conventional technologies, such as Lidar sensors, vision-based systems, and global navigation satellite system (GNSS) devices. Unlike vision-based and Lidar systems, radar is robust to different lighting and weather conditions and maintains performance in fog, darkness, dust, or snow. It is also able to measure the target velocity directly, unlike Lidar which relies on successive position measurements. Furthermore, radar is able to function in GNSS-denied and reception-poor environments, such as indoors, in tunnels, and near obstructions [4].

Odometry is the process of estimating a vehicle’s position over time using data from onboard sensors. Within the field of vehicle odometry, FMCW radar has proven effective for ego-velocity estimation. The ego-velocity of a vehicle is the velocity as measured from its own frame of reference. FMCW radar, by using the Doppler shift of return signals, is able to accurately measure the relative radial velocity of objects. By aggregating the radial velocity measurements it possible to estimate the velocity of the sensor. However, challenges arise in dealing with non-stationary objects, ghosting, and reflections in the environment, which must not be included in velocity estimates. Because of this, much research has recently be given to exploration of techniques for FMCW radar ego-velocity estimation, driven by the growing development of autonomous vehicles [5].

4AISystems¹ is a company specialising in the production of advanced driver advisory systems for rail vehicles. One of their key products is the HORUS system, which is designed to improve rail safety by detecting objects and track infrastructure. The HORUS system uses a sensor assembly, called the SensorBar, which houses an inertial navigation system (INS), three colour cameras and two thermal cameras. Their current approach to odometry uses the INS in combination with an odometer radar and visual odometry. The INS fuses data from GNSS and an inertial measurement unit (IMU) as well as from external sources. It was hoped that the odometer radar could be replaced with the more accurate and versatile FMCW radar.

Accurate odometry is essential for the functionality of HORUS since it is often important to know the trains position relative to objects. For instance, it is necessary to know the distance to signals in order to calculate braking distance and time. Another use of odometry is in determining which regions of the camera input should receive more focus in the object detection network. If track information is known, precise odometry can be used to make crops of the camera images above the tracks in front of the train. Objects located on or near the tracks are the most critical to detect, so increasing their detection accuracy is highly desirable. This makes accurate ego-velocity valuable in the HORUS

¹<https://4aisys.com/>

system for railway safety through improved object detection.

1.2 Problem statement

Accurate ego-velocity estimation is a key challenge in ensuring the reliability and effectiveness of driver advisory systems in rail applications. However, various existing odometry methods have significant limitations impacting their performance in certain conditions.

Wheel odometry estimates velocity by measuring the rotation of a rail vehicle's wheels. This method is prone to inaccuracies due to wheel slippage, such as that occurring under wet conditions or with fallen leaves on the tracks. Slippage can also occur when ascending a hill, during acceleration, or when braking. Furthermore, wheel odometry can become inaccurate over time due to wheel wear and deformation [6].

Vision-based odometry uses visual data from cameras to estimate movement by analysing changes to the scene over time. However, these methods struggle in poor visibility conditions such as at night, in rain, or in fog. Changes in lighting, such as darkness or glare can also degrade visual input and lead to unreliable velocity estimates. This makes visual odometry a good secondary approach but one that cannot be relied upon for all situations.

Inertial navigation systems combine data from satellite positioning systems and from an IMU to estimate position and velocity. The IMU measures acceleration, rotation, and orientation. IMU-based methods suffer from drift over time, as errors from the accelerometer accumulate over time. GNSS is usually employed to correct this drift, however its performance is restricted in areas with poor reception, such as tunnels, valleys, and dense urban areas. In such areas the IMU-drift is allowed to accumulate leading to progressively less accurate velocity and position estimates [7].

Radar odometers measure velocity using radar signals that reflect off the environment to determine velocity. Conventional radar odometers are aimed at the ground and only measure the Doppler velocity of a single target, not accounting for range or direction. This means that such systems can only measure the forward component of the velocity. Moreover, radar odometers require calibration to maintain accuracy and depend on the texture of the ground surface, which can impact measurements.

Given these limitations, there is a need for a reliable and versatile ego-velocity estimation method that can perform well in various conditions. Incorporating FMCW radar ego-velocity is a potential solution to this problem, by maintaining performance in situations where other techniques struggle.

1.3 Objectives

The objectives of the project are the following:

1. To explore the capabilities of a 4D FMCW radar sensor when integrated with the existing sensors.

The FMCW radar output has the potential to be utilised by several existing processes, as well as new ones. Doppler radial velocity measurements may assist with odometry. Object detection functionality may benefit from the added information on target distance, velocity, and RCS.

2. To improve rail vehicle odometry using radar ego-velocity estimation.

By using the target Doppler measurements from a radar sensor, an estimate for ego-velocity may be obtained from the radar. This estimate may then be used by the inertial navigation system to improve the accuracy of odometry.

3. To evaluate and compare ego-velocity estimation algorithms in terms of accuracy and processing time.

Several contemporary ego-velocity algorithms will be implemented and assessed in terms of their performance.

4. To successfully integrate the radar sensor with the HORUS system.

The output from the radar sensor should be read by the system and parsed into a format readable by the other modules within HORUS.

1.4 Scope

The scope of the project included the following:

- **Radar parser** - The radar parser is used to interface between HORUS and the radar hardware. From the raw sensor output it produces a list of points from which ego-velocity can be estimated.
- **Radar visualisation tool** - The visualisation tool will be included in the 4Eyes developer interface and is used to display the output from the radar. Radar targets are projected onto camera images, and are coloured to improve object recognisability.
- **Radar ego-velocity estimation** - Uses the points from the radar to estimate the ego-velocity of the vehicle.

- **Evaluation of ego-velocity estimation algorithms** - Several alternative algorithms were researched and implemented. They were then evaluated against a ground truth to determine their accuracy.

1.5 Significance

This research is significant because it explores odometry techniques for application in the rail industry. Many existing algorithms for radar ego-velocity estimation have been developed and optimised for automotive environments, where operational conditions differ significantly from those in rail systems. By evaluating and modifying these methods for rail applications, this research aims to help improve the effectiveness of driver advisory systems for trains.

Furthermore, current sensor fusion techniques often rely on traditional approaches such as the extended Kalman filter (EKF). In this study a neural-network-based INS is used to perform sensor fusion, aiming to improve accuracy over EKFs.

Additionally, a new algorithm for radar ego-velocity estimation is proposed, combining random sample consensus (RANSAC) with Cauchy robust loss. Incorporating a robust loss function has potential to improve the success rate of RANSAC while maintaining accuracy. This could improve the overall robustness of odometry methods, particularly in situations with large non-stationary objects and environmental noise.

1.6 Outline

This report is organised into six primary chapters, following the introduction. Chapter 2 provides an overview of the existing literature relevant to FMCW radar systems and contemporary radar ego-velocity estimation methods. Chapter 3 details the experimental design, data collection processes, and analysis methods used. Additionally, it covers the system design specifications, software development methodology and ethical considerations. Chapter 4 discusses the system implementation, including the development process, implementation details, evaluation criteria, and testing. Chapter 5 presents and analyses the results of the study, including those of the prototype testing and the Monte Carlo simulation. Chapter 6 provides a summary of the key findings, discusses implications of the research, and provides recommendations for future work.

2

Literature Review

Given the diverse and developing approaches to FMCW radar-based ego-velocity estimation, it was necessary to conduct a systematic review of recent literature. This section outlines the methodology used to conduct the literature review, followed by a summary of the key findings. The review begins with the theoretical background, explores contemporary applications of FMCW radar, and concludes with an assessment of current radar ego-velocity estimation techniques.

2.1 Methodology

The literature review was designed to ensure a systematic and thorough investigation of existing research related to FMCW radar and ego-velocity estimation. The primary research question for the review was:

- How does the integration of FMCW radar to a vision-based and inertial navigation system improve environmental awareness capability in a rail environment?

This question was supported by the following sub-questions:

- **RQ1** - What are the current technological capabilities of frequency-modulated continuous-wave (FMCW) radar systems for ego-velocity estimation?

- **RQ2** - How can FMCW radar sensors be integrated with vision-based systems to improve detection accuracy and reliability under varied environmental conditions?

To identify relevant sources, a targeted search was conducted for articles published between 2016 and 2024 on the databases IEEEExplore, ScienceDirect, and SpringerLink. The time frame was designed to capture recent advancements in FMCW radar and ego-velocity estimation, however, earlier foundational work was also included. These databases were selected due to their focus on technical and scientific papers, which was deemed to be appropriate to the project. Two queries were executed on each of the selected databases. The first query related to RQ1 and was made for items containing the following in their title, abstract, or index terms:

(FMCW OR Automobile Radar OR Millimeter?wave radar)

AND

(Ego?velocity OR Ego?motion OR Self?velocity OR Inertial odometry)

The second query related more closely to RQ2 and was made as follows:

(Radar OR Point cloud OR Millimeter?wave radar OR FMCW)

AND

(Sensor fusion OR Data fusion OR Multi?modal OR Vision?based
OR Thermal imaging)

AND

(Object detect* OR Object tracking OR Object classification OR Safety system
OR Collision avoidance OR Navigation)

The effectiveness of the search was validated by ensuring the inclusion of key works in the results, and that the returned results are relevant to the research questions. Articles not from peer-reviewed journals or conference proceedings were excluded from analysis. Results were then further filtered manually for whether they addressed the research questions. The remaining articles were evaluated based on their methodological rigour and contribution to the field. Additional sources were obtained based on key works referenced in multiple papers. An overall analysis of the final set of literature is given in the following sections.

2.2 Research Findings

Radar is a sensor that uses radio waves to determine the distance, direction, and radial velocity of surrounding objects. Radar technologies may be broadly categorised into either

pulsed radar and continuous-wave radar. Pulsed radar emits periodic pulses and listens for return signals between these. The time delay allows for calculation of the range of objects. In contrast, CW radar simultaneously transmits and receives a radio signal and hence is able to detect many objects at the same time. However, to measure the range of objects, this signal must be modulated in some way. CW radar systems are simpler to manufacture, have low power consumption and, unlike pulsed-radar, have no minimum range. These characteristics have made CW radar desirable in the fields of localisation, odometry, and object detection. Continuous-wave radar has, for these reasons, become ubiquitous in advanced driver-advisory systems as well as finding applications in naval, aerial, and rail navigation [3][8].

The most common method of modulation for continuous-wave radars is frequency modulation. FMCW radar varies the frequency of the signal over a certain period of time, called the chirp time. The received signal reflected from some object is mixed with the transmit signal to determine the difference in frequency. This difference is called the beat frequency and may be obtained with a low-pass filter. This principle, for a single target reflection, is illustrated in Figure 2.1.

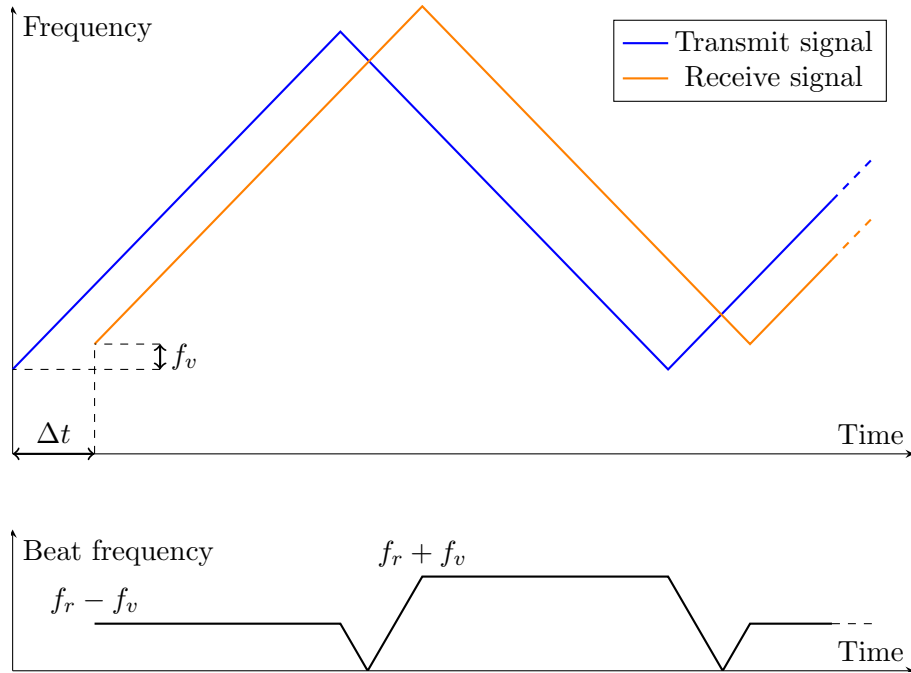


FIGURE 2.1: FMCW radar range and velocity measurement principle. The top plot shows transmit and receive signals for a single target using triangular frequency modulation. The bottom plot shows the corresponding beat frequency, used to estimate range and velocity.

The distance to an object creates a time shift in the received signal (Δt) and hence

a frequency shift (f_r) that increases with range. Additionally, if an object is travelling radially relative to the sensor, the return frequency will also be Doppler-shifted (f_v) [9]. Since both factors affect the received frequency, it is necessary to differentiate between them. To do this a triangle waveform may be used as in Figure 2.1. With a triangle wave, the range frequency shift and the Doppler frequency shift may be disambiguated based on up-ramp beat frequency (b_u) and the down-ramp beat frequency (b_d) as follows:

$$\frac{b_d - b_u}{2} = \frac{(f_r + f_v) + (f_r - f_v)}{2} = f_r \quad (2.1)$$

$$\frac{b_d + b_u}{2} = \frac{(f_r + f_v) - (f_r - f_v)}{2} = f_v \quad (2.2)$$

From the range frequency shift, the range may be obtained based on the speed of light and the frequency modulation rate. From the Doppler frequency shift, the relative velocity may be obtained based on the speed of light and the centre frequency of the signal.

However, this approach fails when returns from multiple objects are received, since each object generates its own set of beat frequencies which must be distinguished. Several methods have been developed to overcome this range-velocity coupling problem, including multiple frequency shift keying (MFSK) [10] and fast chirp modulation [9]. FMCW radar uses interferometry from multiple receive antennae to determine the direction of targets. To maintain high angular resolution with small antennae, a multiple-input multiple-output (MIMO) architecture is often employed [11].

FMCW radar systems have attracted much interest recently due to their low power consumption, low cost, and small form-factor [2]. FMCW radar is able to provide rich data which cannot be obtained from other types of sensors. The range, radial velocity, direction, and RCS of objects can be measured directly. RCS of a target indicates what proportion of incident energy from a signal is reflected back towards the sensor. It is affected by the object's size, material, and shape. FMCW traditionally only measures the azimuth of a target, meaning it provides essentially a birds-eye view and suffers from poor near-range detection [12]. 4D Radar technology additionally has the ability to measure the elevation angle of targets, providing richer data with fewer blind-spots. This is achieved by arraying the antennas in a grid rather than a horizontal line, sacrificing azimuth angular resolution for the additional ability to measure target elevation. Further analysis of radar imaging is made in [11], noting the superior performance in inclement weather conditions compared to lidar and vision-based systems. Radar performs well even in scenes with fog, heavy rain, darkness, or snow. Additionally, it remains functional in GNSS-denied and low-reception environments, such as in tunnels, near obstructions, and indoors [4]. Such properties make radar systems suitable for effective application in ego-velocity estimation,

object detection, and object tracking.

These capabilities have led to FMCW radar becoming commonplace in autonomous automobile object detection systems. However, challenges are presented by the sparsity of typical radar data. To address this, [13] applies a deep neural network to perform inferences on radar data with dilation and erosion pre-processing operations. These morphological operations increase the connectivity of the object features detected by radar, thereby reducing noise and improving detection accuracy. A similar approach is outlined in [14] where a self-attention mechanism is used to extract features from radar data. This approach allows a network to focus on significant features by dynamically adjusting their importance. Processing techniques such as these have allowed greater efficacy of radar systems, making them valuable for improving the reliability of automobile navigation systems.

While the primary application of FMCW radar has been to road vehicles it has seen some adoption in the field of railway safety technology. [15] demonstrates the capability of radar systems to reliably detect rails and potential obstacles up to 400 metres ahead of a rail vehicle. They employed a modified version of the density-based spatial clustering applications with noise (DBSCAN) algorithm. This algorithm treats speed as an additional dimension on top of the three spatial dimensions and thereby groups proximal points into clusters, which are likely to represent a discrete object. The algorithm is robust to outliers which is essential in a rail environment given its strong scattering properties. To account for the sparsity of the radar output, multiple frames are concatenated before being processed. Additionally, greater weight is given to the lateral distances since most points are expected to be arranged in the forward direction. This approach demonstrates that object detection is feasible with FMCW radar output utilised independently.

When radar has been employed in conjunction with vision-based sensors, approaches have typically employed a convolution neural network (CNN) to fuse camera and radar data. [16] introduces a rail corridor intrusion detection system using a CNN in this way. The approach to fusion involves dynamically adjusting the contribution of the radar and camera data based on the scenario. For example, in poor lighting the radar would receive greater weighting while in clear conditions the camera data would be weighted more heavily. Crucially, their method achieves the real-time performance necessary for deployment in active railway. Complimentarily, [17] uses the Mask-R-CNN algorithm to detect hazards on train lines. Their algorithm first detects the rail tracks to define a region of interest before subsequently detecting objects within this area, helping to avoid false positives from adjacent objects. This algorithm has demonstrated high precision and performance in real-world conditions.

Several approaches for FMCW radar ego-velocity estimation have been established in recent years with several variations of each. Techniques typically taking the Doppler velocity of stationary targets and ignoring moving targets and noise. Widely-researched approaches for this include the random sample consensus (RANSAC) algorithm, clustering methods, robust loss functions, normal distribution transform (NDT), and deep learning-based techniques.

RANSAC is an iterative algorithm for estimating the parameters of a model from a dataset with many outliers. In each iteration a subset of points are chosen randomly and a model is fit to them. This model is then evaluated against the entire dataset to distinguish inliers, which fit the model well, from outliers, which do not. This process is repeated over many iterations, and the model with the most inliers is retained. This model is finally fit with the complete set of inliers to improve its accuracy. Due to its robustness against outliers, this algorithm has proved effective in ego-velocity estimation. The method for fitting the model is conventionally a linear least squares (LLSQ) regression, as used successfully in [4], [18], [19]. However, alternative loss functions have been explored, including orthogonal distance regression (ODR), as used by [20], [21]. ODR uses the perpendicular distance from the model rather than only the y-axis distance as in LLSQ. This may improve the accuracy of the loss, however, [21] found that it provided no significant improvement over LLSQ. Aside from this, there have been few attempts at investigating the use of RANSAC with alternative loss functions for radar ego-velocity estimation.

Clustering methods have also been researched for ego-velocity estimation. They aim to group targets based on their spatial proximity and relative velocity, to identify all stationary points. In [22], the DBSCAN clustering algorithm is used to identify stationary objects and filter out moving objects. This approach was able to achieve success, but struggled to accurately measure the lateral velocity. The approach relies on the assumption that the largest cluster of objects corresponds to static objects, so is likely to fail if the stationary objects do not form a single cluster.

Another class of approaches employed is normal distribution (NDT)-based techniques as used by [23], [24]. NDT is a probabilistic method where a point cloud is divided into cells, each of which is assigned a normal distribution representing the probability of sampling a point at a given spatial coordinate. Then, from a second point cloud, a transform is found which maximises the likelihood of the second point cloud arising from such a distribution. Unlike other methods mentioned, NDT-based algorithms require point clouds from two different times in order to compute a transformation, introducing a short delay depending on the cycle time of the radar.

Robust loss functions are another technique explored for radar ego-velocity estimation.

Using robust loss aims to reduce the impact of outliers without using explicit outlier-rejection such as RANSAC. They do this by assigning lower weight to points that deviate significantly from the model, as opposed to LLSQ which assigns these greater weight. Common robust loss functions include Huber loss, Cauchy loss and Tukey's biweight function. In [25], [26] the Cauchy loss function is used to perform velocity estimation by reducing the impact of environmental noise and clutter. This function is particularly harsh on outliers, which is suitable for the high-reflectivity of railway scenes. Robust regression has found to be successful in radar ego-velocity estimation, however, the loss functions often require parameter tuning to balance sensitivity to outliers with accuracy for inliers.

Deep learning-based techniques have recently emerged as a powerful approach to radar ego-motion estimation. In [5], [27], [28] a neural network is implemented for velocity estimation, trained on either raw radar data or pre-processed features. Such techniques may be classified as either image-based, voxel-based, or point based. Approaches often use a single architecture to fuse radar multi-modal inputs, such as that from radar, cameras, lidar, and inertial measurement units. While deep learning approaches have been found to outperform other methods in certain situations, they usually require large datasets for training and can be computationally expensive.

3

Methodology and Design

This chapter outlines the methodological approach and design considerations of this project. A structured research experiment was designed to provide a reliable and replicable evaluation of the implemented FMCW radar-based ego-velocity estimation techniques. In particular, the FMCW radar-based estimate was to be compared with an estimate produced by the existing system. This section describes the experiment methodology including its design, data collection processes, analysis methods, and validation methods. An overview of the software development methodology, system architecture, and ethical issues is also given.

3.1 Research Design

The research design for this project employed both experimental and simulation-based techniques to evaluate different radar-based ego-velocity estimation algorithms.

An experiment was designed using two physical prototypes to collect data at low and high velocities respectively. The experiment compared several alternative odometry methods as follows:

- Current system: INS + odometer radar
- Cauchy loss

- RANSAC + Linear least squares (LLSQ)
- RANSAC + Cauchy loss

The current approach, using an INS and odometry radar was used as a baseline against which to compare the radar-based techniques. It was hoped that at low-velocities and with good GNSS reception, this system would be sufficiently accurate to serve as a ground truth. However, this assumption was found to be incorrect and so an additional simulation was required to be performed using the RadarScenes dataset.

For each radar cycle, the velocity estimate produced by each of the four methods was recorded. From this two evaluation metrics were calculated: mean error (ME) in velocity and mean absolute error (MAE) in velocity. ME was used to detect any systematic biases in the velocity estimates. A non-zero ME indicates the presence of a consistent overestimation or underestimation by an algorithm. MAE was used to quantify the overall accuracy of the velocity estimates by comparing them to ground truth values.

3.2 Data Collection

Two physical prototypes were designed to gather real-world sensor data - a handcart was to be used for low-velocity testing and a car for high-velocity testing. However due to hardware availability constraints for mounting the sensors, the high-velocity prototype testing could not be conducted within the project time frame. As a result, the publicly available RadarScenes dataset was used to simulate high-velocity scenarios. The prototype design and data collection processes are detailed in the following sections.

3.2.1 Low-Velocity Prototype

The aim of the low-velocity prototype is to evaluate the performance of the ego-velocity estimation algorithms with the FMCW radar sensor at low speeds. A ground truth was obtained using the velocity produced by the INS in conjunction with the odometer radar. The two estimates were then be compared to determine error of the radar and algorithm.

The low-velocity prototype consisted of several sensors mounted on a wheeled cart. The sensors were connected to an Ethernet switch, which is also connected to a computer. All components were to be powered by a portable battery. Figure 3.1 depicts the relationships between components of the prototype. Figure 3.2 shows an annotated photograph of the assembled prototype.

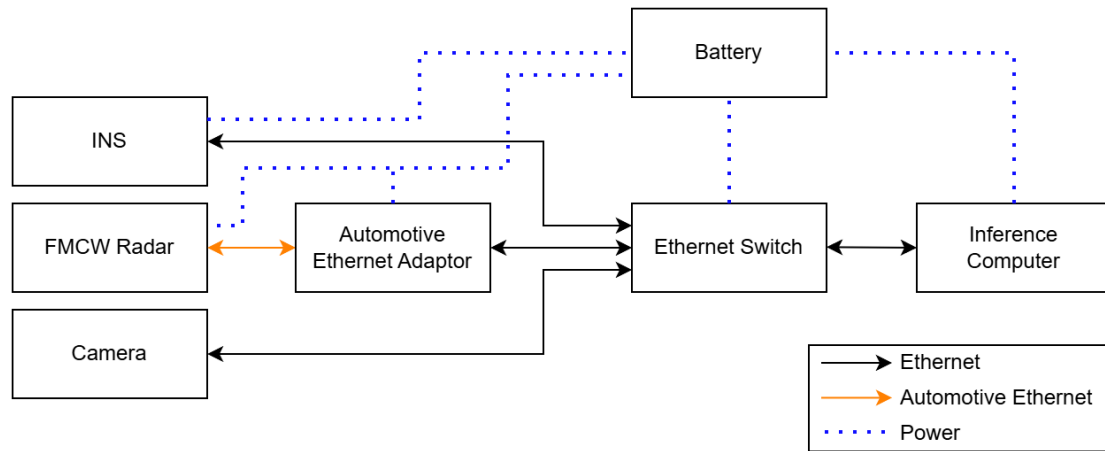


FIGURE 3.1: Design of the low-velocity prototype, showing the connections between components. Note that the camera uses power over Ethernet and so is not directly connected to the battery.

The included components are as follows:

- **FMCW radar** - The radar sensor which collects radar returns from many objects. This data is used in the ego-velocity estimation algorithms.
- **Camera** - The camera is used for a visual verification of radar data and to assist with analysis.
- **INS** - The Inertial navigation system determines positioning of the unit based on the detected acceleration, GNSS positioning, and heading. The heading is determined based on the relative positions of the two GNSS antennae.
- **Odometer radar** - The odometer radar measures the Doppler velocity of the vehicle in the forwards direction based on the movement of the ground. This velocity is used by the INS to improve position estimation.
- **Battery** - A portable battery powers the Ethernet switch, FMCW radar, INS unit and odometer radar. This allows the trolley to be used on an outdoor course.
- **Computer** - The computer performs the role of the inference computer used in the live system. For the low-velocity prototype it additionally records data from the FMCW radar, Camera and INS (including the odometer radar velocity).
- **Ethernet switch** - The switch creates a network connecting the sensors to the computer, enabling data transfer between them.

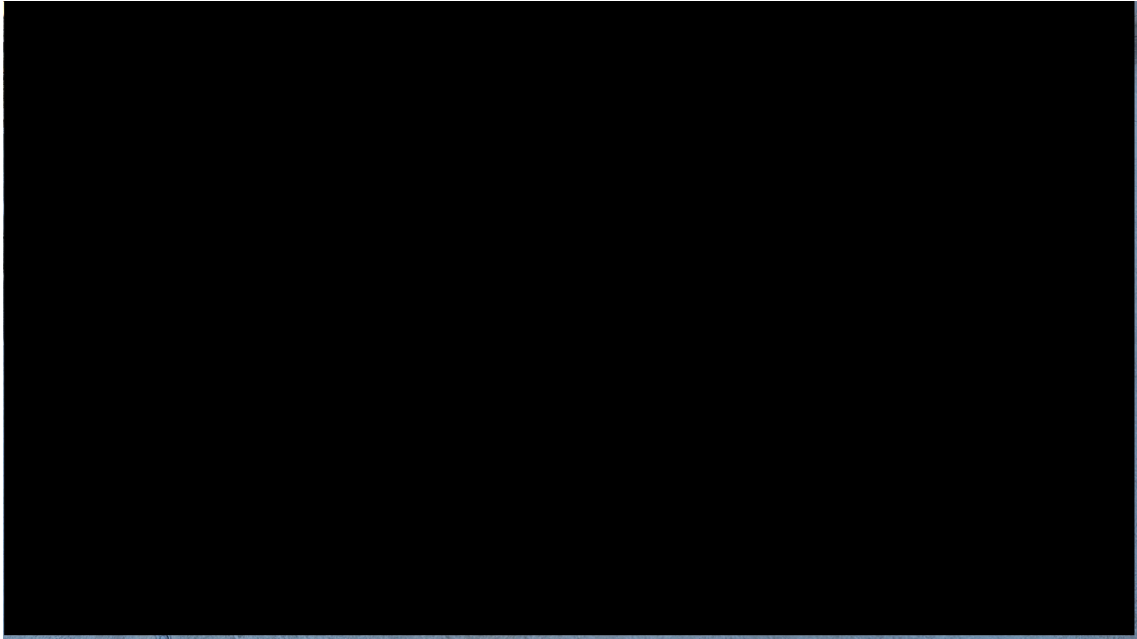


FIGURE 3.2: Photograph of the implemented low-velocity prototype with labelled components. The Ethernet switch and Automotive Ethernet Adaptor are positioned inside the cart and are not visible.

- **Automotive Ethernet adaptor** - The automotive Ethernet adaptor is used to interface between the radar sensor, which uses the automotive Ethernet protocol, and the Ethernet switch, which uses standard Ethernet.

Details on specific components used for in implementation of the low-velocity prototype are listed in Section 4.1.3.

The assembled prototype was to be manually taken on a short circuit around the paved driveway and car park adjacent to the office building. The test lasted for approximately three minutes and involved remaining stationary, travelling in a straight line, turning, and reversing. During this time objects passed into the radar's field of view including cars and pedestrians.

The data collected by this prototype included: point cloud messages from the radar, odometry messages from the INS, and images from the camera. This information was recorded by the system so that it could later be replayed and analysed.

3.2.2 High-Velocity Prototype

The high-velocity prototype was designed to evaluate the performance of ego-velocity estimation algorithms at speeds comparable to those of live deployment. The prototype

consisted of the SensorBar with the full set of sensors mounted on a car. Ultimately, testing with this prototype could not be completed due to unavailability of the SensorBar hardware. As a result of this, the testing of the algorithms at high-velocities could only be completed in a simulated environment.

3.2.3 RadarScenes Dataset

The RadarScenes dataset is a publicly available dataset designed to support radar-based perception research. It contains point cloud data collected from four radar sensors mounted on a vehicle. The dataset contains several hours worth of driving in various conditions, including urban and rural environments, and has ground truth optometry values for ego-velocity. The scenes often containing objects such as cars, pedestrians, and cyclists. This made it useful for evaluating the accuracy of ego-velocity estimation algorithms [29].

The database contains position, velocity, RCS, and timestamp data from the radars stored in HDF5 format. A Python script was written to convert and store the HDF5 files as Protocol Buffer messages. This allowed the RadarScenes data to be replayed and analysed in the same way as the output from the low-velocity prototype. This dataset was valuable in assessing the ego-velocity estimation since the high-velocity prototype could not be used. It also allowed testing in multiple conditions and with real-world objects and clutter.

For this analysis a set of sequences was selected from the RadarScenes dataset, covering a variety of environmental conditions including weather, traffic volume, and road surface. The chosen sequences for the simulation are listed in Table 3.1.

Index	Surface	Weather	Traffic	Distance (m)	Max Velocity (m/s)
10	Smooth	Cloud	High	1443	13.9
67	Smooth	Clear	Little	668	13.9
89	Bumpy	Rain	No	752	12.9
108	Smooth	Clear	No	2281	19.6
138	Smooth	Clear	Medium	700	12.4

TABLE 3.1: Selected RadarScenes environmental characteristics

3.3 Data Analysis

Once data had been collected, it was replayed into the system with each of the ego-velocity estimation algorithms. The algorithmic data was synchronised with the ground truth using their associated timestamps. When an ego-velocity timestamp fell between

two ground truth timestamps, linear interpolation was used to determine an intermediate value. For each algorithm the mean error and mean absolute error from the ground truth were calculated.

To determine if any of the algorithms performed significantly better than the others, the following hypotheses were tested:

- **Null Hypothesis (H_0):** There is no significant difference in the performance of the three ego-velocity estimation algorithms.

$$H_0 : \mu_1 = \mu_2 = \mu_3$$

where μ_1 , μ_2 , and μ_3 are the MAE values for each of the algorithms.

- **Alternative Hypothesis (H_1):** At least one of the ego-velocity estimation algorithms performs significantly differently.

$$H_1 : \exists i, j \text{ such that } \mu_i \neq \mu_j \quad \text{for } i \neq j$$

To test these hypotheses, an analysis of variance (ANOVA) was conducted on the MAE across the radar scene sequences, to determine whether there were any significant differences between the MAE values of the three algorithms.

3.4 Validation and Verification

To ensure the robustness of research methodology, a validation and verification process was conducted. Each software component was unit tested to ensure that the underlying algorithms functioned correctly. This helped to resolve potential bugs which may have introduced error into the research and impacted reliability.

The methodology aimed to incorporate data from both the INS and RadarScenes dataset. By using these independent datasets, the research approach was validated by ensuring that findings were consistent and not reliant on a single source. This was further strengthened by the inclusion of multiple ego-velocity estimation algorithms. This allowed the verification of these against the current system to better determine whether the radar data improved upon the existing system.

Additionally, when testing with the RadarScenes dataset, it was ensured that repeated measurements were made under different environmental conditions. This helped to verify that the system was robust under various circumstances.

3.4.1 Uncertainty Quantification

A Monte Carlo simulation was carried out to evaluate the uncertainty of outputs from each algorithm. A data recording was modified to simulate measurement noise based on the performance data of the radar, listed in Appendix A. The noise was sampled from a normal distribution with a mean of zero and a standard deviation corresponding to the listed sensor performance values. The estimation process was repeated 1000 times, each with different noise values. For each iteration the standard deviation was calculated to represent the impact of measurement noise on the final estimates. The aggregated results for this simulation are shown in Table 3.2.

TABLE 3.2: Monte Carlo simulation results for ego-velocity estimation algorithms

Algorithm	Velocity standard deviation (ms^{-1})		
	x	y	z
RANSAC + LLSQ	0.0061	0.0174	0.1902
Cauchy	0.0063	0.0180	0.1856
RANSAC + Cauchy	0.0058	0.0177	0.2014

The results of the simulation were consistent across the velocity estimation methods indicating that none of the processes was disproportionately affected by noise. Additionally, it was found that the uncertainty in the x-velocity was the lowest, followed by the y-velocity, while the z-velocity showed the greatest variance. One possible reason for this is that the elevation field of view, at 20 degrees, was the smaller than the azimuth field of view, which was 100 degrees. This led to a greater spread of data along the y-axis compared to the z-axis. Additionally, the x-axis data had the widest spread, since it extended the full range of the radar. This provided a larger interval on which the regression could be performed, thereby reducing the standard deviation of the estimates.

3.5 Design Specifications

The system uses a component-based architecture with asynchronous messaging between modules, to allow for reusability and extensibility. A high-level system architecture diagram for the system shown in Figure 3.3. Only components developed as part of this project are been depicted.

The radar parser listens for UDP packets from the radar and creates Protocol Buffer message containing a point cloud. This module includes the SMSTP parser which handles the specific protocol used by the radar device. The point cloud message is handled by a message bus which distributes it to all subscribers, currently the point cloud visualisation

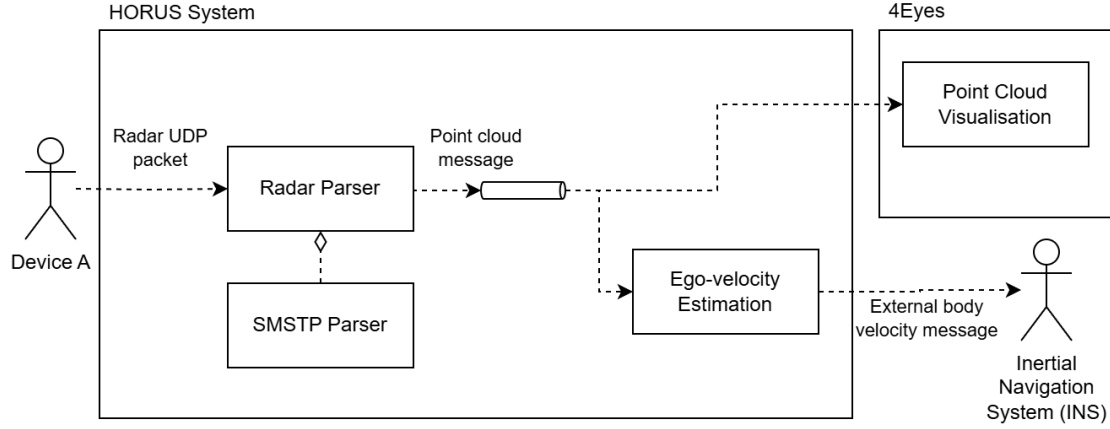


FIGURE 3.3: System architecture diagram for radar-based ego-velocity estimation. Other components of the HORUS and 4Eyes systems have been omitted for simplicity.

tool and the ego-velocity estimation module. The ego-velocity module uses a point cloud message to make an estimate of the velocity of the vehicle. This estimate is then emitted as an external body velocity message, which is used by the INS and fused with GNSS and IMU measurements. Details on the implementation of each of the modules are given in Section 4.2.

Using a component-based architecture provided several benefits in future development of the system. The modules are loosely-coupled so that additional consumers or producers of the point cloud message could be added to the system without modifying any of the existing components. For example, if object detection capability were to be added it could simply subscribe to the point cloud message to use it in any processing. Additionally, the visualisation and velocity estimation components would continue to work correctly with a different point cloud producer, such as if the radar were replaced with a lidar sensor. The system also has the capability to accommodate multiple radars however some changes would need to be made depending on how the output from each of the radars would be aggregated.

3.6 Software Development Methodology

The development methodology used in the radar integration project was an Agile approach modified to suit the research-driven nature of the project. This methodology was chosen to manage the likely changes to requirements and to foster collaboration with the client. Consequently, regular working versions of the software were to be produced, enabling

ongoing client feedback and iterative testing.

The Agile methodology was adapted somewhat in order to accommodate the dynamic requirements typical of a research-driven project. In this project, analysis of existing research was decided to be critical in determining the nature of the final system to be built as well as its requirements. As a result, research for the project was conducted simultaneously with development activities. Each Agile iteration incorporated research findings in addition to testing results from the previous iteration.

The structure of this approach involved two-week sprints. Each sprint began with planning based on tasks completed, research findings, and client feedback from the previous sprint. From this review, development and research tasks were identified to be prioritised for the upcoming sprint period. Meetings with the project client were conducted at least once per sprint. This process allowed the project to remain adaptable and ensure changes to requirements were addressed.

This methodology was chosen for its flexibility in accommodating changes to requirements arising both from the client's needs and research findings. In initial consultation with the client, it was determined that while they had considered many potential applications for the radar, not all of them could realistically be implemented in the time-frame of the project. Additionally, it was not clear which aspects would be feasible or effective, and research and testing was required to understand this better. Furthermore, since the solution was to be developed primarily by an individual, a lightweight methodology was most appropriate. For these reasons, a modified Agile approach was selected.

Alternative methodologies were considered but found to be less suitable for the project. The waterfall model, which uses sequential phases was ill-suited due to its inflexibility for the changing requirements of this project. Scrum, while agile, typically requires team meetings with a scrum master directing development; this structure was unreasonable given the project being undertaken by an individual. Similarly, methodologies like Lean, which emphasises waste reduction, was unnecessary for such a small project. Other approaches such as prototyping, feature-driven and test-driven development each have merits but require more resources or are more rigid than the modified agile methodology.

3.7 Ethical Considerations

The collection of radar data is generally considered to pose a low risk, however, it still raises several minor ethical concerns. The use of cameras in the prototype may inadvertently capture individuals or their property without their knowledge. As a result, it was ensured that only public spaces were recorded by the camera. Additionally, the recorded data was

only used internally and for the purposes of this project. Access to the data was limited only to personnel involved in the research.

While the research project does not directly involve rail vehicles, the findings could later be applied to real-world safety systems. As a result there is a responsibility to ensure the system is tested thoroughly for robustness and reliability.

4

Implementation and Evaluation

This chapter describes the implementation details of the radar-based ego-velocity estimation system, the development process, and the tools used. It also gives an overview of some challenges encountered during development and how they were addressed. Finally, the evaluation criteria and testing methods used to assess the system's performance are presented.

4.1 Development Process

The development of the system followed an Agile approach, allowing for continuous feedback and improvement. Several programming languages, frameworks, and libraries were used in development, as detailed in the following sections.

4.1.1 Languages

The following languages were used in the system implementation:

- **C++** was the primary language used in implementation, chosen for its efficient execution and compatibility with the existing system. It provided the performance necessary for the real-time nature of the system while offering abstraction for easier development.

- **Python** was used for the radar configuration due to its simplicity and flexibility, since speed is less critical in this area.

4.1.2 Frameworks and Libraries

The following frameworks and libraries were used in the system implementation:

- **NUClear** is a C++ framework designed for real-time modular systems. The framework is designed to prioritise the fast routing of messages between modules [30]. This was used the development of the system as it greatly simplified asynchronous messaging.
- **Eigen** is a C++ library for linear algebra, including matrices, vectors, numerical solvers, and related algorithms [31]. It was used in the system to efficiently handle operations with matrices and vectors, including transformation matrices. It is also used to perform least squares regression.
- **NLopt** is a library for nonlinear optimisation written in C and with an API directly callable from C++ [32]. It is used to minimise the Cauchy loss function when performing ego-velocity estimation. This is further described in Section 4.2.6.

4.1.3 Hardware

A number of hardware components were used in the development and testing of the implementation. The function of each is given in Section 3.2.1. The specification for each component used was as follows:

- **Radar sensor** - [REDACTED]
- **Automotive ethernet adaptor** - [REDACTED]
- **INS** - [REDACTED]
- **Camera** - [REDACTED]
- **Ethernet switch** - [REDACTED]
- **Odometer radar** - [REDACTED]

4.1.4 Development Environment

The development environment was set up to facilitate efficient development and ensuring compatibility with the system infrastructure. Development was conducted on a Linux-based machine to ensure that it could be deployed on the inference computer in the future. All dependencies were managed using Docker to ensure consistency and simplify deployment across environments. Git was employed for collaboration and managing versions, with a feature-based branching strategy. CMake was used to handle build configurations and Clang was the compiler used. Visual studio code was used for development, with extensions for C++, Python, Docker, CMake, and Git among others.

4.1.5 Coding Practices

The development process adhered to several coding practices to ensure the system was robust and maintainable. Git was used for version control, providing a traceable history of code changes aiding in management of the progress of the software. Feature branching was employed whereby new features were developed on an isolated branch, ensuring experimental changes didn't affect the main branch. Prior to merging any code, a review was required to be performed by another developer. Furthermore, any code to be merged was required to be unit tested and achieve 80% statement coverage.

Automatic formatting tools were used to enforce a consistent coding style across the company. This improves code readability and reduces conflicts during merges.

Documentation was required for each module, including a description, usage details as well as a list of each message type the module consumes or emits. Additionally, logging was implemented at appropriate levels of verbosity (i.e., trace, debug, info, warn, error, fatal). The display of logging levels may be toggled to facilitate monitoring and debugging.

4.2 Implementation Details

This section provides an explanation of the components and algorithms developed to perform radar-based ego-velocity estimation. It details the configuration and parsing of radar data, the visualisation tool, and the mathematical techniques employed for velocity estimation.

4.2.1 Radar Configuration

The radar configuration tool is used to read or set parameters on the radar sensor using a command-line interface. In particular, it is used to specify the source and destination IP

addresses that the radar should use. This means that the data and diagnostic messages from the radar will be sent to the correct IP address - usually that of the inference computer. This tool is to be run when the system starts to ensure that the correct settings are being used for the radar.

4.2.2 SMSTP Packet Parser

This module is responsible for receiving data packets from the radar sensor and reinterpreting them into a format understandable by other modules.

The SMSTP parser functions as a state machine, with the state depending on current position within an SMSTP packet. The parser keeps extended state variables of the current buffer index and the last index of a segmented packet. A state machine for the parser is shown in Figure 4.1.

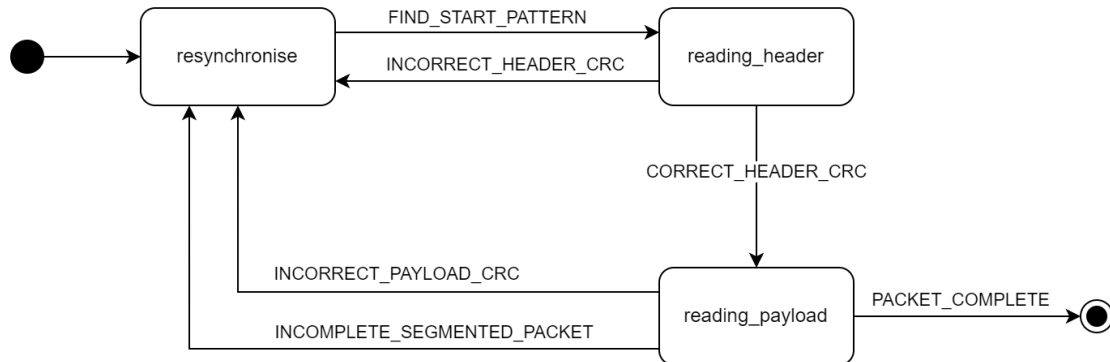


FIGURE 4.1: State diagram illustrating the process flow of the SMSTP parser.

The function of each state is as follows:

1. **Resynchronise** - This is the entry state when a new packet is to be parsed. It is also reached when a packet is found to be invalid and is interrupted. While in this state all bytes are ignored unless they form a packet start pattern. Finding a start pattern triggers a transition to the *reading header* state.

2. **Reading header** - In this state the header length is read from the header and the parser will wait for enough bytes for a complete header. Once a complete header is read, the header Cyclic Redundancy Check (CRC) is computed and checked against the value contained in the header. If this is correct, the the parser returns to the *reading payload*, otherwise it returns to the *resynchronise* state.
3. **Reading payload** - In this state the payload length is read from the header and the parser will wait for the entire payload. If a payload CRC is included in the packet, this is verified and if it is incorrect the parser returns to the *resynchronise* state. If the SMSTP packet is segmented and not all segments have arrived, the parser returns to the *resynchronise* state. If the packet is not segmented or all segments have been read, then an *SMSTPPacket* Protocol Buffer message is emitted containing the complete packet and the parser state is reset. The contents of this message is provided in Appendix B.1.

4.2.3 Point Cloud Visualisation

The point cloud visualisation tool is used for debugging the radar output and processing. The module listens for point cloud Protocol Buffer messages (See Appendix B.1) and displays the data from the most recent of them through the 4Eyes web interface. The number of point clouds displayed is configurable and controls how long points take to disappear. For each camera known, the points of the point cloud are projected to the reference frame of that camera and then onto the corresponding camera image. This is possible because each image message also contains a transformation matrix to world space.

Furthermore, a radar front-view and a radar top-down-view are produced. Since the elevation field of view for the used radar sensor is quite small (20°), the top-view is often useful in visually identifying objects and their positions. Additionally, the displayed points are given a colour based on their absolute radial velocity and their RCS. This enables the easy distinction between moving objects, which typically have a large absolute radial velocity, stationary objects, which often have a large RCS, and clutter, which generally has small values for both. The radar views are shown in Figure 4.2.

If new camera images are received, the position of the points are adjusted to match the new world-space transforms. This is useful since the radar produces point clouds at a rate of about 20Hz but cameras often have a much faster frame rate. Additionally, since the point clouds are usually quite sparse, several frames are combined and displayed at once to get a better idea of where objects are. The number of point clouds persisted is configurable for the visualisation tool.

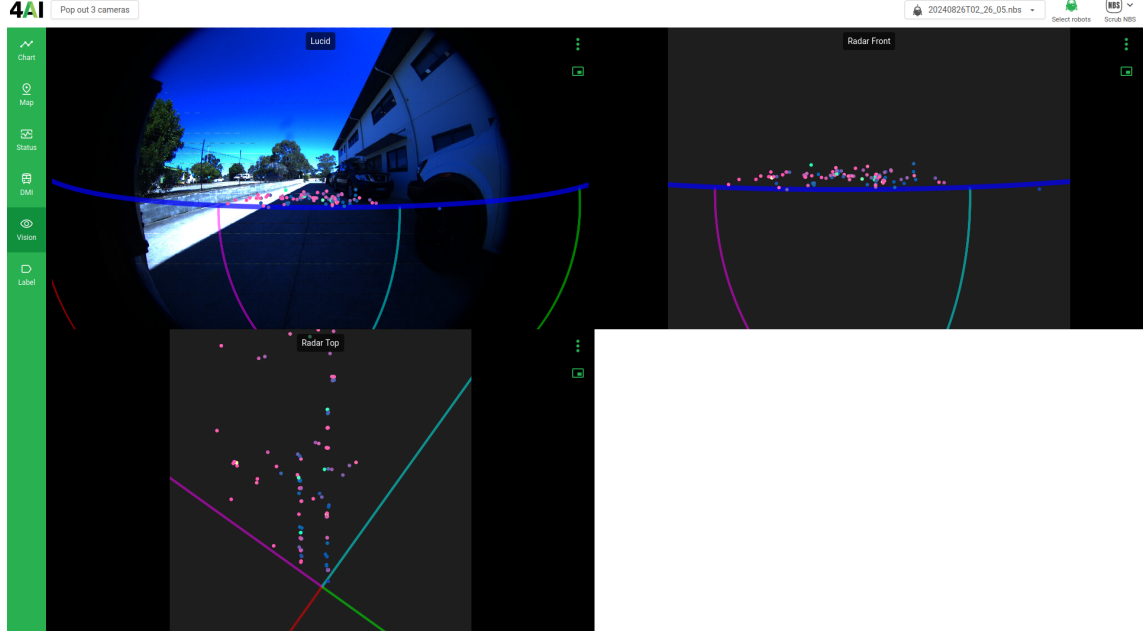


FIGURE 4.2: Point cloud visualisation tool view including a camera view (top-left), the radar front-view (top-right), and the radar top-down-view (bottom-left).

4.2.4 Least-Squares Radar Velocity Estimation

Ego-velocity estimation may be performed with a Doppler radar based on the measured Doppler velocity of radar targets. The radar sensor uses the Doppler-shift of the return signal to accurately determine the relative radial speed of targets. However, it is not possible to directly calculate a target's velocity vector since only the radial component of the velocity is measured. Because of this, targets at differing angles must be used to reconstruct the sensor velocity (v_s).

The measured Doppler radial velocity (v_r) is the vector projection of the true target velocity (v) onto the direction vector of the target.

$$v_r = d \cdot v_s = d_x v_x + d_y v_y + d_z v_z \quad (4.1)$$

Given the azimuth angle (θ) and elevation angle (ϕ) of a target, the components of its

direction vector are obtained as follows:

$$\begin{aligned} d_x &= \cos \phi \cos \theta \\ d_y &= \cos \phi \sin \theta \\ d_z &= \sin \phi \end{aligned}$$

Given a set of N measurements with radial velocity v_{r_i} and direction vector v_{d_i} , and applying (4.1) yields:

$$\underbrace{\begin{bmatrix} v_{r,1} \\ \vdots \\ v_{r,N} \end{bmatrix}}_b = \underbrace{\begin{bmatrix} d_{x,1} & d_{y,1} & d_{z,1} \\ \vdots & \vdots & \vdots \\ d_{x,N} & d_{y,N} & d_{z,N} \end{bmatrix}}_A \underbrace{\begin{bmatrix} v_{sx} \\ v_{sy} \\ v_{sz} \end{bmatrix}}_x \quad (4.2)$$

A velocity estimate may be determined from a set of points using a linear least squares (LLSQ) regression. Thus a solution for (4.2) may be obtained as follows:

$$x = (A^T A)^{-1} A^T b \quad (4.3)$$

The solution to (4.3) using all radar measurements will not provide an accurate indication of the velocity since it assumes all targets to be static. In reality some targets will be moving, and some will be false detections due to environmental noise and reflections. Such outliers must be removed, and the estimate made only on stationary targets. This is described in the following section. Finally, the resulting velocity must be negated to give the ego-velocity estimate in sensor-space; $v_s = -x$.

4.2.5 RANSAC Outlier Rejection

The Random Sample Consensus (RANSAC) algorithm, as first proposed in [33], is used to detect only stationary targets in the radar output by ignoring outliers. The relative velocity of only these stationary point then represents the negation of the ego-velocity of the radar.

The RANSAC algorithm performs a number of iterations in each of which a random subset of the data is selected. In this implementation three points are chosen as this is the minimum required to solve (4.3). A model is then fitted to the selected points to determine a hypothetical sensor velocity, as outlined in section 4.2.4. Next, all data points are tested against the fitted model and all that fit the model are regarded as the consensus set (i.e., the set of inliers). For each iteration the number of inliers is recorded, and the

largest consensus set is retained. After all iterations, the model is improved by re-fitting it to the entire consensus set.

In this process, the consensus set should reflect the stationary targets in the environment and fitting with least squares gives the ego-velocity of the sensor. The assumption is made that there is no group of targets with the same velocity and having more data points than there are stationary points.

Since RANSAC is non-deterministic there is no guarantee that it will return a useful result. However, increasing the number of algorithm iterations (k) improves the probability of success (p). It is considered that RANSAC is successful if in at least one iteration when it chooses n points, these are all inliers (in this case, stationary points). The probability of selecting an inlier (w) may be approximated as:

$$w = \frac{(\text{number of inliers})}{(\text{number of total points})} \quad (4.4)$$

Given this, the probability of RANSAC succeeding can be modelled by [33]:

$$k = \frac{\log(1 - p)}{\log(1 - w^n)} \quad (4.5)$$

The value of n is set to 3 since this is the number of unknowns in the least squares regression, as per (4.2). If two-dimensional RANSAC were to be performed n would instead be set to 2. The proportion of inliers, w , was determined to be approximately 0.7. Setting the desired success probability, p to 0.995. Using these values with (4.5) gives a value for k of 12.6. Therefore, using the RANSAC algorithm with 13 iterations gives a probability of success greater than 99.5%.

4.2.6 Cauchy Robust Loss

As an alternative to RANSAC with LSSQ, robust regression techniques were investigated. Unlike the traditional least squares approach, which assumes a normal distribution and is sensitive to outliers, the Cauchy loss function assumes a Cauchy distribution of data. This makes it more suitable for data containing strong outliers, potentially removing the need to use RANSAC to explicitly reject outliers [34]. The scaled Cauchy loss function is given by:

$$L = \frac{c^2}{2} \log \left(1 + \left(\frac{y - \hat{y}}{c} \right)^2 \right) \quad (4.6)$$

where $c \in (0, \infty)$ is a scaling constant, $\hat{y}$ is the predicted value of v_r and y is the measured value. The term $y - \hat{y}$ is the residual. For small residuals, the function resembles a mean

squared error (MSE) loss function, while for larger residuals it resembles a logarithmic function. This reduces the influence of outliers compared to inliers.

The Cauchy loss function for different scale factors is shown in Figure 4.3. The parameter c determines where the function transitions from being quadratic to being logarithmic. Smaller values of c make the function less sensitive to large residuals, reducing the impact of outliers. Typically, c is chosen to be proportional to the mean absolute deviation (MAD) of the distribution.

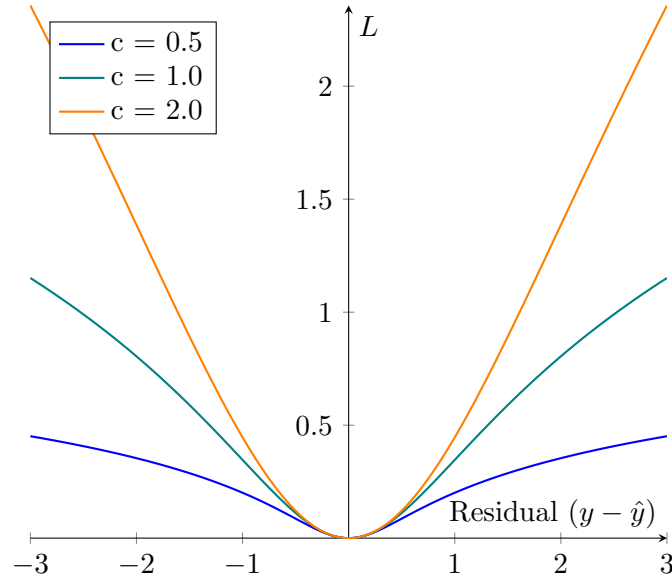


FIGURE 4.3: Cauchy loss function for different scaling factors c .

In the application of ego-velocity estimation, a regression is performed with a Cauchy loss function. This approach helps to reduce the influence of strong outliers, such as those multi-path reflections and moving targets.

To solve the optimisation problem, the Method of Moving Asymptotes (MMA) is employed, as described in [35]. Since this method is a gradient-based method, the gradient of the Cauchy loss is required. By differentiating the loss function, (4.6), with respect to the predicted value $\hat{y}$, the gradient function may be obtained:

$$\frac{dL}{d\hat{y}} = \frac{y - \hat{y}}{1 + \frac{(y - \hat{y})^2}{c^2}} \quad (4.7)$$

4.3 Challenges and Solutions

During implementation of the project, several technical and logistic challenges were encountered. One major issue was the unavailability of the SensorBar hardware, which was initially planned to be used for live data collection and system testing. The SensorBar contained the various sensors necessary for this testing, all of which were precisely calibrated. While testing with a rail vehicle was not possible during this project, the SensorBar could also be mounted on a car. It was hoped that by mounting the radar in a SensorBar the radar output could be precisely aligned with camera images. This would allow the association of points with objects identified by the detection network, providing potential training data for radar-based object detection. However, partway through the project it was determined that collecting aligned data with the SensorBar and car would not be possible until near the end of the project. This was all SensorBars that currently available were required to be assembled and shipped to clients in the near future. To address this, the project scope was adjusted to focus on ego-velocity estimation rather than on object detection and tracking. As another workaround for this issue, a handcart was constructed and used as a low-velocity substitute for the test car. However, without the SensorBar this prototype could not provide the alignment precision necessary for object identification.

After shifting the focus of the project to ego-velocity estimation, it was still planned that the car be used for testing when a SensorBar was available. Unfortunately however, one never became available and so live testing at higher velocities was not possible. To overcome this issue, a simulation was instead performed using a publicly available radar dataset.

Another significant issue was the lack of thorough documentation for the radar sensor's output message format. This led to some delays, as trial and error was required to interpret data and configure the radar correctly. The radar manufacturer did provide a piece of software to interact with the radar, but no success was found with this software as the protocol appeared to be different entirely. Instead a plugin for the packet analysing software Wireshark had to be used to decode the radar packets. This workaround provided sufficient understanding of the packet structure to successfully implement the custom radar parser.

When first testing the low-velocity prototype outdoors, the radar generated a much larger number of data points compared to indoor use. This increase in data led to the radar packets being segmented, which the initial implementation did not handle properly, causing data points to appear in incorrect positions. This issue was addressed by introducing a state machine in the SMSTP parser. Using a state machine provided a more robust way

to manage the complexity of parsing segmented packets, with a different state being used depending on how far the system was through reading a message. Additionally, the packet parser was designed as an isolated module within the system's architecture, following the single responsibility principle. This ensured that changes to the parser did not affect other parts of the system, making the software easier to maintain and extend in the future.

4.4 Evaluation Methods

In evaluating the project implementation, structured testing was used in conjunction with more informal feedback from the project client. Since the system did not include an interface for the end user, the client was the most important source of input. During development, the client was continuously consulted to discuss implementation design decisions and methods to ensure that the project aligned with their goals. In addition to this, more formal meetings were held bi-weekly to receive feedback, and decide on the next steps in implementation.

The testing strategy adopted a phased approach to ensure each stage of development was thoroughly validated. Types of testing conducted included unit testing, integration testing, system testing, and performance testing.

The first components to be developed was the radar parser and visualisation tool. Testing was performed on the parser to ensure that it produced well-structured point cloud messages and was able to handle edge cases such as packet segmentation and invalid CRC checks. Integration testing was performed with the radar sensor as well as between the parser and visualiser. This verified that the point cloud message and message bus interface worked correctly. Following this, initial system testing was performed with the sensor, parser and visualisation in the office, with the radar mounted in a fixed location. With this set up, the visualisation was verified by passing objects in front of the radar and checking they appeared in the point cloud view. Additionally, the projection was verified by checking that the distance to objects was consistent with physical measurements.

Once the basic system was working correctly, the ego-velocity estimation algorithms were implemented. Unit testing was performed on each algorithm and integration testing was performed between them and the radar parser. Finally, system testing was performed with the complete set of sensors using the low-velocity prototype. The radar ego-velocity was compared with the odometry produced by the other sensors. Performance testing was also conducted, measuring for velocity algorithm the time taken to process a large, fixed number of point cloud messages. Soak testing was also part of this, leaving the system running continuously for several days to ensure its long-term stability. This testing

verified that the system could handle extended operation without failure, necessary for safety-critical applications. Results for the testing of the algorithms are given in chapter 5.

5

Results

This chapter presents the findings from the low-velocity testing and RadarScenes simulation, which were conducted to evaluate the performance of the implemented radar-based ego-velocity estimation algorithms. The results are analysed to determine their accuracy, bias, and how they compare to existing odometry methods. The following sections detail the outcomes of these experiments, along with a comparison to existing literature and an assessment of the study's limitations.

5.1 Low-Velocity Testing

Initial testing of the ego-velocity estimation algorithm was performed using the low-velocity prototype (See Section 3.2.1). The radar ego-velocity measurement was compared to the current system using the INS data. A comparison of the forward velocity over time for the two approaches is shown in Figure 5.1.

It was hoped that the INS data could be used as a ground truth against which to compare the radar estimate, however, it was found to be unsuitable for this purpose. The data showed negative velocities in places as well as sections with velocities of unexpected magnitudes. Since this did not correspond to the testing conducted, it could not be used as a ground truth. Despite this, the low-velocity prototype testing still provided some

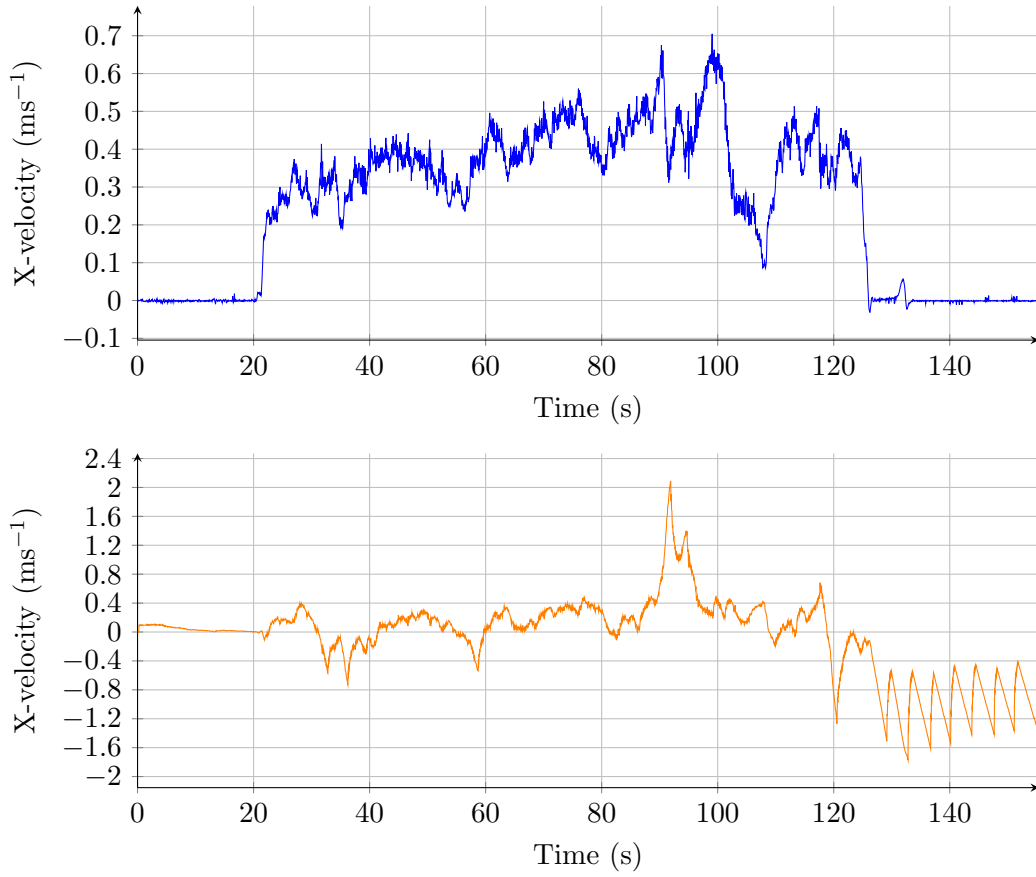


FIGURE 5.1: Comparison of radar ego-velocity estimate (above) and INS velocity estimate (below). Note the difference in y-axis scales.

useful insights. The overall shape of velocity over time from the two sources matches well, indicating that they are indeed correlated albeit weakly.

The features of the radar data appear to arrive slightly earlier than the odometry from the INS. This is most likely because the INS performs more processing on the data before outputting it, leading to a very small delay. It was also found that the radar data appeared noisier, however, it is unclear to what extent this noise reflects actual data, as the handcart prototype experienced some bumpiness.

Visual inspection of the chart indicated that the radar-based ego-velocity is more accurate than the current system when tested with the low-velocity prototype. The INS data displayed a range of speeds that included significantly negative values, which is incorrect, as the cart was never pulled backward. Additionally, towards the end of the testing, the velocity fluctuated considerably, likely due to poor GNSS reception. Consequently, it was

concluded that the radar velocity estimate provided notable improvement over the INS-based estimate for measuring low velocities. However, to accurately assess the performance of the algorithm, the RadarScenes simulation was deemed necessary.

5.2 RadarScenes Simulation

Further testing of the ego-velocity estimation algorithm, along with comparison against some existing standard algorithms was performed using the RadarScenes dataset (See Section 3.2.3). The results of this simulation is shown in Table 5.1.

Sequence	Algorithm	ME (m/s)	MAE (m/s)
10	Ransac + LSSQ	0.087	0.136
	Cauchy	-0.006	0.153
	Ransac + Cauchy	0.079	0.132
67	Ransac + LSSQ	0.028	0.057
	Cauchy	0.005	0.074
	Ransac + Cauchy	0.033	0.063
89	Ransac + LSSQ	0.021	0.055
	Cauchy	0.018	0.083
	Ransac + Cauchy	0.025	0.065
108	Ransac + LSSQ	0.148	0.187
	Cauchy	-0.012	0.155
	Ransac + Cauchy	0.068	0.149
138	Ransac + LSSQ	0.007	0.067
	Cauchy	-0.003	0.084
	Ransac + Cauchy	0.006	0.068

TABLE 5.1: Performance Metrics for Ego-Velocity Estimation Algorithms Across RadarScenes Sequences

Comparing results across sequences, RANSAC with least squares generally appeared to provided the lowest absolute error, except for in sequence 108 where RANSAC with Cauchy loss performed the best. The Cauchy robust loss algorithm generally had the greatest mean absolute error values of the three algorithms. Despite this, while the Cauchy approach maintained had mean error values close to zero, indicating no systematic bias. In contrast, both RANSAC with least squares and RANSAC with Cauchy loss consistency produced positive mean error values, suggesting the precense of a systematic overestimation in these algorithms.

The results of the simulation showed that the performance of each algorithm was dependent on the sequence, with significant variability between them. Sequence 108 showed the highest MAE values, indicating that it was likely the most challenging of the set, while sequence 138 showed the lowest values, indicating it was the least challenging.

Following further statistical analysis using a two-way ANOVA test, it was observed that there was no statistically significant difference in the mean error between the algorithms ($p = 0.074$). Additionally, the analysis found no statistically significant difference in the mean absolute error between the algorithms ($p = 0.61$). However, the sequence was found to have a very significant impact on MAE ($p = 0.0001$), indicating that the variability in algorithm performance more strongly influenced by the sequence than the choice of algorithm. This highlights the large impact that environmental conditions, such as the number of moving objects in a scene, has on ego-velocity estimation accuracy. Based on the results of the ANOVA test, the null hypothesis - which states there is no difference in mean absolute error between the estimation algorithms - could not be rejected.

Figure 5.2 shows the X-velocity estimates over time as obtained by the RANSAC with Cauchy algorithm. The cyan line represents the radar-based estimate and the red line shows the ground truth.

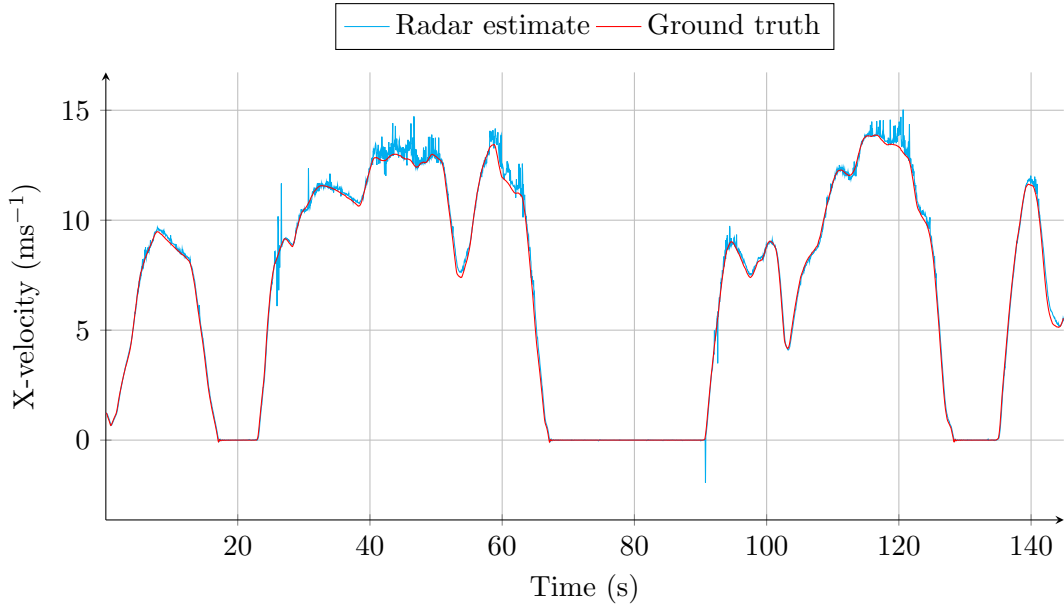


FIGURE 5.2: RadarScenes sequence 10 chart showing radar ego-velocity estimate from the RANSAC with Cauchy algorithm.

For most of the sequence, the radar-based estimate closely follows the ground truth, particularly in the lower velocity regions. However, at higher velocities the radar estimate

has more significant deviations, and more frequently overshooting than undershooting the true values. This pattern contributes to the observed positive mean error value for this algorithm.

5.3 Contributions and Literature Comparison

The integration of FMCW radar into the multi-sensor system demonstrated a significant improvement in odometry accuracy at low velocities. However, further analysis is needed to confirm whether this holds true for higher velocities.

The performance of the RANSAC algorithm aligns with the accuracy values reported in prior research, including studies such as [4] and [27]. The ability of RANSAC and Cauchy methods to mitigate errors caused by environmental noise and dynamic objects has been successfully replicated.

The novel contribution of this work is the use of RANSAC with a robust loss function, the Cauchy loss function, to remove outliers in radar data. This method did not consistently outperform RANSAC with linear least squares.

Overall, the results show that radar ego-velocity estimation is a valuable addition to sensor systems and driver advisory systems are likely to benefit from improved odometry.

5.4 Limitations

There were several limitations in the evaluation of the radar-based ego-velocity estimation algorithms. The low-velocity testing found no reliable ground truth meaning that a measure of the accuracy of the estimates could not be assessed. Furthermore, no live testing at higher velocities could be performed. Being able to do this would have more closely approximated live deployment conditions and allowed for a better comparison.

The testing with the RadarScenes dataset used a relatively small sample size, with only 15 data points. The statistical power would have been improved with a greater sample size, making a detection of difference between the algorithms more likely if there is something to detect. Additionally, the algorithms were not tested in live environmental conditions, such as in rain or darkness, which would have provided a more comprehensive analysis of their performance and reliability under adverse conditions.

Another limitation was related to the hardware used: the radar sensor has maximum unambiguous velocity 33 ms^{-1} . This limitation could cause problems if the system is deployed in scenarios where the ego-vehicle is travelling at high speeds, since it is possible for rail vehicles to exceed this speed. As a result, a different sensor with a greater velocity

range would be required in such situations.

6

Conclusions

The goal of this research was to explore the potential of FMCW radar for ego-velocity estimation in rail vehicles. It aimed to address the limitations of traditional odometry methods, which often struggle in environments with poor GNSS reception, darkness, or inclement weather. By integrating FMCW radar with an existing multi-sensor system, this project aimed to improve the accuracy and reliability of driver advisory systems in rail applications.

In the course of this project, software was developed successfully to facilitate the use of FMCW radar sensor. This included a radar parser, which efficiently decoded and structured raw radar data into point clouds. These point clouds could be visualised with a tool integrated into the 4Eyes developer interface. Ego-velocity estimation was also implemented to perform real-time velocity estimation based on the radar output. The overall system was designed in a modular fashion, allowing for reusability and laying a foundation for future extension.

The project successfully demonstrated the application of 4D FMCW radar for ego-velocity estimation. The findings demonstrate that radar-based ego-velocity estimation provides a significant improvement over the current HORUS system, particularly at low-velocities. In such scenarios, the radar approach was more robust, maintaining accuracy

where the INS data fused with odometry radar failed due to GNSS signal loss. Several techniques for radar-ego-velocity estimation were implemented including RANSAC with least squares, Cauchy robust loss, as well as a novel approach using RANSAC with Cauchy loss. This method, while not consistently outperform traditional approaches, offers a promising direction for future development. It would be beneficial to evaluate these methods on a larger dataset to determine if an increased statistical power reveals a more consistent performance difference.

The study had several limitations. The low-velocity testing with the handcart prototype did not yield a reliable ground truth, limiting the ability to properly quantify the accuracy of the radar estimate in a live environment. Additionally, high-velocity testing could not be performed due to a prolonged unavailability of the SensorBar hardware. The RadarScenes simulation provided useful insights but did not faithfully replicate the a live railway environment, since the data was collected on roads and lacked scenarios containing severe weather or nighttime conditions.

The implications of this research are significant in the development of robust and reliable driver advisory systems for rail vehicles. The role of radar in such systems has been reinforced by demonstrating their high accuracy. By integrating radar data, rail safety systems can better handle adverse conditions where other sensors fail. The insights gained from this study may inform future designs for radar-based ego-velocity estimation and sensor fusion algorithms, facilitating more resilient railway safety systems.

To build on this work, future research should prioritise high-speed testing in real-world conditions, using a properly calibrated SensorBar mounted to a vehicle. Additionally, advanced sensor fusion techniques to integrate radar data with INS and vision systems should be investigated. Furthermore, future work could explore using the rich data obtained by 4D FMCW radar for object detection and tracking in rail environments. Integrating radar with vision-based systems could allow more reliable detection of obstacles, especially in poor visibility conditions. Techniques to fuse these data sources could be developed to more accurately classify and track objects, using the radar's measurements of distance, radial speed, and RCS.

In conclusion, this research has shown success in integrating FMCW radar into a railway driver advisory system, offering a more accurate and resilient method for ego-velocity estimation. Nevertheless, there is still potential for refinement in the techniques used and many possibilities for broader applications of radar data to further improve railway safety.

References

-
- [2] J. Hasch, E. Topak, R. Schnabel, T. Zwick, R. Weigel, and C. Waldschmidt, “Millimeter-wave technology for automotive radar sensors in the 77 ghz frequency band,” *IEEE Transactions on Microwave Theory and Techniques*, vol. 60, no. 3, pp. 845–860, 2012.
 - [3] A. Venon, Y. Dupuis, P. Vasseur, and P. Merriaux, “Millimeter wave fmcw radars for perception, recognition and localization in automotive applications: A survey,” *IEEE Transactions on Intelligent Vehicles*, vol. 7, no. 3, pp. 533–555, 2022.
 - [4] J. Barra, T. Creuzet, S. Lesecq, G. Scorletti, E. Blanco, and M. Zarudniev, “Micro-drone ego-velocity and height estimation in gps-denied environments using an fmcw mimo radar,” *IEEE Sensors Journal*, vol. 23, no. 3, pp. 2684–2692, 2023.
 - [5] P. R. M. de Araujo, A. Noureldin, and S. Givigi, “Toward land vehicle ego-velocity estimation using deep learning and automotive radars,” *IEEE Transactions on Radar Systems*, vol. 2, pp. 460–470, 2024.
 - [6] J. Yi, J. Zhang, D. Song, and S. Jayasuriya, “Imu-based localization and slip estimation for skid-steered mobile robots,” in *IEEE International Conference on Intelligent Robots and Systems*, pp. 2845–2850, 10 2007.
 - [7] K. Fukutoku, H. Masuda, and T. Murakami, “Internal sensor based kinematic parameters estimation using acceleration/deceleration motion,” *2021 IEEE International Conference on Mechatronics (ICM)*, pp. 1–6, 2021.
 - [8] S. A. S. Mohamed, M.-H. Haghbayan, T. Westerlund, J. Heikkonen, H. Tenhunen, and J. Plosila, “A survey on odometry for autonomous navigation systems,” *IEEE Access*, vol. 7, pp. 97466–97486, 2019.

- [9] Z. Tong, R. Renter, and M. Fujimoto, “Fast chirp fmcw radar in automotive applications,” in *IET International Radar Conference 2015*, pp. 1–4, 2015.
- [10] J. He, R. Zhang, W. Sheng, Y. Han, and X. Ma, “An improved mfsk waveform for low-cost automotive radar,” in *2016 CIE International Conference on Radar (RADAR)*, pp. 1–5, 2016.
- [11] S. Briskén, F. Ruf, and F. Höhne, “The recent evolution of automotive imaging radar and its information content,” *IET Radar, Sonar & Navigation*, vol. 12, 04 2018.
- [12] G. Li, Y. L. Sit, S. Manchala, T. Kettner, A. Ossowska, K. Krupinski, C. Sturm, and U. Lubbert, “Novel 4d 79 ghz radar concept for object detection and active safety applications,” in *2019 12th German Microwave Conference (GeMiC)*, pp. 87–90, 2019.
- [13] M. Sabry, A. Hussein, A. Elmougy, and S. Abdennadher, “Augmented radar points connectivity based on image processing techniques for object detection and classification,” in *Proceedings of the 17th International Conference on Computer Vision Theory and Applications (VISAPP)*, pp. 535–542, 01 2022.
- [14] B. Xu, X. Zhang, L. Wang, X. Hu, Z. Li, S. Pan, J. Li, and Y. Deng, “Rpfa-net: a 4d radar pillar feature attention network for 3d object detection,” in *2021 IEEE International Intelligent Transportation Systems Conference (ITSC)*, pp. 3061–3066, 09 2021.
- [15] W. Pan, X. Fan, H. Li, and K. He, “Long-range perception system for road boundaries and objects detection in trains,” *Remote Sensing*, vol. 15, p. 3473, 07 2023.
- [16] T. Ye, X. Zhang, Y. Zhang, and J. Liu, “Railway traffic object detection using differential feature fusion convolution neural network,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, pp. 1375–1387, 2021.
- [17] I. Mahmud, J. Shin, C. Mistry, Y. Tomioka, and M. Ph. D., “Advancing wildlife protection: Mask r-cnn for rail track identification and unwanted object detection,” *IEEE Access*, vol. PP, pp. 1–1, 01 2023.
- [18] Y. S. Park, Y.-S. Shin, J. Kim, and A. Kim, “3d ego-motion estimation using low-cost mmwave radars via radar velocity factor for pose-graph slam,” *IEEE Robotics and Automation Letters*, vol. 6, no. 4, pp. 7691–7698, 2021.

- [19] D. Kellner, M. Barjenbruch, J. Klappstein, J. Dickmann, and K. Dietmayer, "Instantaneous ego-motion estimation using doppler radar," in *Proceedings - IEEE International Conference on Robotics and Automation*, pp. 869–874, 10 2013.
- [20] A. Kingery and D. Song, "Improving ego-velocity estimation of low-cost doppler radars for vehicles," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 9445–9452, 2022.
- [21] D. Kellner, M. Barjenbruch, J. Klappstein, J. Dickmann, and K. Dietmayer, "Instantaneous ego-motion estimation using multiple doppler radars," in *2014 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1592–1597, 2014.
- [22] M. Hoffmann, L. Krabbe, C. Schüßler, P. Gulden, and M. Vossiek, "Instantaneous ego-motion estimation using a coherent radar network," in *2022 19th European Radar Conference (EuRAD)*, pp. 321–324, 2022.
- [23] P.-C. Kung, C. Wang, and W.-C. Lin, "A normal distribution transform-based radar odometry designed for scanning and automotive radars," *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 14417–14423, 2021.
- [24] M. Rapp, M. Barjenbruch, M. Hahn, J. Dickmann, and K. Dietmayer, "Probabilistic ego-motion estimation using multiple automotive radar sensors," *Robotics and Autonomous Systems*, vol. 89, pp. 136–146, 2017.
- [25] A. Kramer, C. Stahoviak, A. Santamaria, A.-a. Agha-mohammadi, and C. Heckman, "Radar-inertial ego-velocity estimation for visually degraded environments," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 5739–5746, 05 2020.
- [26] V.-J. Štironja, L. Petrovic, J. Peršić, I. Marković, and I. Petrovic, "Rave: A framework for radar ego-velocity estimation," *2024 IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems (MFI)*, 06 2024.
- [27] S. Zhu, A. Yarovoy, and F. Fioranelli, "Deepego: Deep instantaneous ego-motion estimation using automotive radar," *IEEE Transactions on Radar Systems*, vol. 1, pp. 166–180, 2023.
- [28] H.-W. Cho, S. Choi, Y.-R. Cho, and J. Kim, "Deep complex-valued network for ego-velocity estimation with millimeter-wave radar," in *2020 IEEE SENSORS*, pp. 1–4, 2020.

-
- [29] O. Schumann, M. Hahn, N. Scheiner, F. Weishaupt, J. Tilly, J. Dickmann, and C. Wöhler, “RadarScenes: A Real-World Radar Point Cloud Data Set for Automotive Applications,” Mar. 2021.
 - [30] T. Houliston, “NUCclear.” <https://nuclear.readthedocs.io/en/latest>, 2016. Accessed: 2024-09-20.
 - [31] G. Guennebaud, B. Jacob, *et al.*, “Eigen v3 documentation.” <http://eigen.tuxfamily.org>, 2010.
 - [32] S. G. Johnson, “The NLOpt nonlinear-optimization package.” <https://github.com/stevengj/nlopt>, 2007.
 - [33] M. A. Fischler and R. C. Bolles, “Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography,” *Commun. ACM*, vol. 24, p. 381–395, jun 1981.
 - [34] T. Mlotshwa, H. van Deventer, and A. S. Bosman, “Cauchy loss function: Robustness under gaussian and cauchy noise,” in *Artificial Intelligence Research* (A. Pillay, E. Jembere, and A. Gerber, eds.), (Cham), pp. 123–138, Springer Nature Switzerland, 2022.
 - [35] K. Svanberg, “A class of globally convergent optimization methods based on conservative convex separable approximations,” *SIAM Journal on Optimization*, vol. 12, pp. 555–573, 2002.

Appendices

[illegible]

B. Protocol Buffer Messages

Protocol buffers are used for asynchronous communication between modules within the system. The following sections outline the messages relevant to ego-velocity estimation.

B.1 Point Cloud

The PointCloud protocol buffer message is assembled from by the radar parser module and emitted. The message may then be consumed by other modules including the radar visualisation and any ego-velocity estimation algorithms. The format is also used to store data recordings as `.nbs` files to be later replayed. The PointCloud message is defined in Listing B.1.

Listing B.1: Protocol Buffer definition used for point cloud data

```
[REDACTED]
```

```
    [
      double x;
      double y;
      double z;
      double roll;
      double pitch;
      double yaw;
      double vx;
      double vy;
      double vz;
    ]
```

B.2 External Body Velocity

The external body velocity message is created by the ego-velocity estimation algorithm. It is used to calculate odometry by the INS, fusing the velocity with GNSS and IMU measurements. The ExternalBodyVelocity message is defined in Listing

Listing B.2: Protocol Buffer definition for ego-velocity estimate

```
message ExternalBodyVelocity {
  double vx = 0.0;
  double vy = 0.0;
  double vz = 0.0;
  double ax = 0.0;
  double ay = 0.0;
  double az = 0.0;
  double roll = 0.0;
  double pitch = 0.0;
  double yaw = 0.0;
}
```